



OpceanAI

A sea of possibilities

Doki

A Complete Container Ecosystem
for Android

Technical Report — Version 0.9.3

OpceanAI Contributors

June 9, 2026

Preface

This technical report provides a comprehensive description of the Doki container ecosystem, covering its architecture, components, and design decisions. It is intended for developers, system administrators, and researchers interested in container technology on Android platforms.

Doki is developed by OpceanAI and released as open-source software under the Apache License, Version 2.0. The project began as a response to the absence of robust containerization tools for Android environments, where traditional solutions such as Docker are unavailable due to kernel limitations and the lack of root access.

This report covers version 0.9.3 of Doki, which includes DokiLink-Lite peer-to-peer mesh networking, 244 CLI commands, 12 isolation levels, and support for both ARM64 and ARMv7 architectures.

OpceanAI | June 9, 2026

Contents

Preface	3
1. Executive Summary	1
1.1. What is Doki?	1
1.2. The Problem	1
1.3. Key Achievements	1
1.4. Impact	2
1.5. Document Structure	2
2. Introduction	3
2.1. Background	3
2.2. The Mobile Containerization Problem	3
2.3. The Doki Approach	4
2.4. Design Principles	4
2.5. Contributions	5
2.6. Document Organization	5
3. System Architecture	6
3.1. Overview	6
3.2. Daemon Architecture	6
3.2.1. Startup Sequence	6
3.2.2. State Management	7
3.3. CLI Architecture	7
3.3.1. Command Structure	7
3.3.2. Flag Parsing	8
3.4. Communication Protocol	8
3.5. Container Lifecycle	9
3.6. Isolation Runtime Selection	9
4. Subsystems and Components	10
4.1. Image Management	10
4.1.1. Image Storage Layout	10
4.1.2. Image Pull Process	10
4.1.3. Search and Registry Interaction	10
4.2. Container Runtime	11
4.2.1. Proot Runner	11
4.2.2. Namespace Runner	11
4.2.3. Native Runner	12
4.3. Dokifile Builder	12
4.3.1. Supported Instructions	12

4.3.2.	Layer Cache	12
4.4.	Compose Engine	12
4.4.1.	Service Dependencies	14
4.4.2.	Profile Filtering	14
5.	Security and Isolation	15
5.1.	Isolation Framework	15
5.2.	Isolation Levels	15
5.3.	Seccomp Filtering	15
5.3.1.	Default Deny List	15
5.3.2.	Custom Profiles	15
5.4.	Linux Capabilities	17
5.5.	AppArmor	17
5.6.	User Namespaces	17
5.7.	Isolation Level Selection	17
6.	Networking	19
6.1.	Network Architecture	19
6.2.	Bridge Networking	19
6.2.1.	Port Mapping	20
6.3.	Internal DNS	20
6.3.1.	Port Selection	20
6.3.2.	DNS Resolution Flow	20
6.4.	DokiLink-Lite Mesh Networking	21
6.4.1.	Encryption Layers	21
6.4.2.	DokiLink Sequence Diagram	21
6.4.3.	Peer Discovery	22
6.4.4.	Gossip Protocol	22
6.5.	Rootless Networking (pasta)	22
6.6.	IPv6 Support	22
6.6.1.	Reverse DNS	22
6.6.2.	Dual-Stack Configuration	23
7.	Storage	24
7.1.	Storage Drivers	24
7.2.	Overlay Filesystem	24
7.2.1.	Layer Structure	24
7.3.	Image Layer Management	24
7.3.1.	Layer Deduplication	25
7.4.	Volume Management	25
7.4.1.	Volume Types	25
7.5.	Filesystem Probes	26
7.5.1.	Detection Probes	26
8.	Development and Build System	27
8.1.	Build Process	27
8.1.1.	Compilation Targets	27

8.1.2. Android ARMv7 Build Fix	27
8.2. Dokifile Build System	28
8.2.1. Build Pipeline	28
8.3. Compose Orchestration	28
8.3.1. Compose File Structure	28
8.3.2. Service Dependency Resolution	29
9. Evaluation and Metrics	30
9.1. Evaluation Methodology	30
9.2. Container Startup Time	30
9.3. Memory Usage	30
9.4. Network Throughput	31
9.5. Image Layer Extraction	31
9.6. Binary Size	32
9.7. Summary	32
10. Future Work and Roadmap	33
10.1. Short-term (v0.10)	33
10.2. Medium-term (v0.11)	33
10.3. Long-term (v1.0)	34
10.4. Feature Priority Matrix	34
10.5. Development Timeline	34
11. Conclusions	36
11.1. Summary of Contributions	36
11.2. Key Metrics	36
11.3. Limitations	36
11.4. Future Impact	37
A. Appendix	40
A.1. A. Complete CLI Command Reference	40
A.2. B. Configuration Reference	40
A.3. C. Environment Variables	41
A.4. D. Release Assets	41
A.5. E. Glossary of Terms	42
A.6. F. Error Codes Reference	42

List of Figures

1.1.	High-level overview of the Doki ecosystem components.	2
3.1.	Doki system architecture. The daemon selects the appropriate runtime based on available host capabilities.	6
3.2.	Container lifecycle state machine. States are shown as rectangles; transitions are labeled with the triggering command.	9
3.3.	The 12 isolation levels available in Doki, ordered from strongest (microVM) to weakest (native). The daemon automatically selects the strongest available level.	9
4.1.	Image pull pipeline. Layers are downloaded in parallel when multiple layers are present.	10
4.2.	Example Compose service dependency graph. Services are started in topological order.	14
6.1.	Doki networking components: bridge network, DNS server, and DokiLink mesh.	19
6.2.	Network topology showing the bridge, containers, port mapping, and NAT.	19
6.3.	DNS resolution flow with cache and TCP fallback.	20
6.4.	DokiLink-Lite connection establishment sequence.	21
7.1.	Overlay filesystem layer structure. Multiple read-only image layers are merged with a writable container layer.	25
8.1.	Build pipeline from source to release archive.	27
8.2.	Dokifile build pipeline with layer cache optimization.	28
8.3.	Compose service dependency graph with start order.	29
9.1.	Container startup time comparison across five base images. Doki's proot runtime adds approximately 100–150 ms overhead compared to Docker on a server.	30
9.2.	Memory usage vs. number of concurrent containers. Doki's proot overhead adds approximately 15 MB per container compared to Docker.	31
9.3.	Network throughput with different encryption configurations. TLS 1.3 adds approximately 5–8% overhead; NaCl secretbox adds an additional 3–5%.	31
10.1.	Doki development roadmap timeline with key milestones.	34

List of Tables

3.1. API endpoint examples and their HTTP methods.	8
4.1. Runtime comparison: proot vs. native runner.	12
4.2. Supported Dockerfile instructions.	13
5.1. Doki isolation levels with their mechanisms and applicable environments. .	16
7.1. Storage driver comparison.	24
7.2. Filesystem probes and their detected environments.	26
8.1. Build targets and their output binaries.	27
9.1. Image extraction times for common base images.	32
9.2. Binary sizes for v0.9.3 release.	32
10.1. Feature priority matrix for the Doki roadmap.	35
11.1. Summary of Doki v0.9.3 key metrics.	37
A.1. CLI command categories and counts.	40
A.2. Daemon configuration keys.	41
A.3. Environment variables recognized by Doki.	41
A.4. Glossary of technical terms used in this report.	42
A.5. Common error codes and their meanings.	42

1. Executive Summary

1.1. What is Doki?

Doki is an open-source container ecosystem designed for Android and Linux environments. Unlike traditional container runtimes that require root access and specialized kernel features, Doki operates entirely in user space using **proot** for syscall interception and filesystem virtualization. The system consists of four binaries—**doki** (CLI), **dokid** (daemon), **doki-compose** (orchestration), and **doki-init** (initialization)—that together provide a complete container lifecycle management solution in approximately 28.3 MB of statically linked Go binaries.

1.2. The Problem

Android presents unique challenges for containerization. The Linux kernel underlying Android restricts access to critical system calls, the **iptables** subsystem is typically unavailable without root, and SELinux policies block common networking operations such as binding to port 53. Traditional container runtimes like Docker and containerd are designed for server environments and cannot operate under these constraints.

The mobile development community lacks a tool that provides Docker-like functionality on Android devices. Developers working with containers on phones and tablets must either use simplified chroot environments or rely on cloud-based solutions that introduce latency and connectivity dependencies.

1.3. Key Achievements

Doki v0.9.3 delivers the following capabilities:

- **12 isolation levels** — from WASM sandboxes to hardware-level microVMs, with automatic detection of the strongest mode available on the host.
- **244 CLI commands** — covering container management, image operations, networking, orchestration, and mesh connectivity.
- **DokiLink-Lite mesh networking** — peer-to-peer encrypted communication using TLS 1.3 with optional NaCl secretbox payload encryption.
- **Rootless operation** — full functionality without root access, using **proot** for syscall interception and **fuse-overlayfs** for filesystem virtualization.

- **Docker Compose compatibility** — support for Dockerfile syntax, Docker Compose files, and BuildKit-style multi-stage builds.
- **Multi-architecture support** — ARM64 and ARMr7 binaries for Android, Linux, and macOS (Darwin).

1.4. Impact

Doki enables developers to run containerized workloads on Android devices without root access, cloud connectivity, or specialized hardware. This opens new possibilities for:

- **Mobile development environments** — running compilers, linters, and test suites directly on phones.
- **Edge computing** — deploying containerized services on mobile devices at the network edge.
- **Education** — teaching container concepts using widely available Android hardware.
- **Field operations** — running isolated tools in environments without traditional server infrastructure.

1.5. Document Structure

This report is organized as follows. [chapter 3](#) describes the overall system architecture. [chapter 4](#) details the individual subsystems. [chapter 5](#) covers the 12 isolation levels. [chapter 6](#) explains the networking stack and DokiLink-Lite. [chapter 7](#) describes image storage and layer management. [chapter 8](#) covers the build system and Dockerfile support. [chapter 9](#) presents performance benchmarks. [chapter 10](#) outlines future development plans.

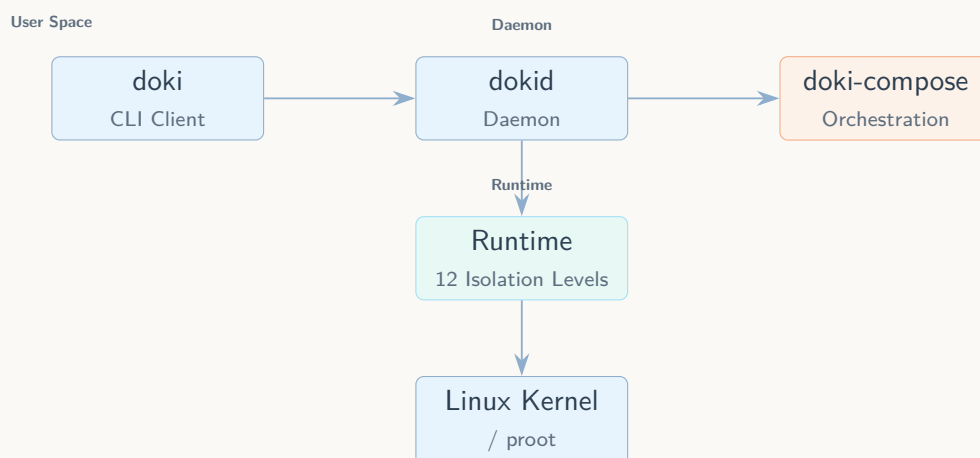


Figure 1.1.: High-level overview of the Doki ecosystem components.

2. Introduction

2.1. Background

Container technology has transformed how software is developed, deployed, and operated. Since the introduction of Docker in 2013, containers have become the standard unit of packaging applications, providing isolation, reproducibility, and portability across computing environments. The container ecosystem has expanded to include orchestration platforms [1], image registries, runtime specifications [3], and networking primitives that collectively enable modern cloud-native architectures.

However, this ecosystem was designed for server-class hardware running Linux distributions with full kernel access. Mobile devices, which represent the majority of computing hardware deployed worldwide, have been largely excluded from the container revolution. Android, which powers over 3 billion active devices [6], presents fundamental barriers to containerization that existing tools cannot address.

2.2. The Mobile Containerization Problem

Android devices face several constraints that prevent traditional container runtimes from functioning:

1. **Restricted syscalls.** The Android kernel disables or restricts numerous system calls required by container runtimes, including `clone()` with certain namespace flags, `pivot_root()`, and `mount()` for bind mounts. SELinux Mandatory Access Control (MAC) policies further limit what operations are permitted.
2. **No root access.** Modern Android devices use verified boot chains and dm-verity to prevent unauthorized modifications. Gaining root access typically requires unlocking the bootloader, which voids warranties and may compromise device security.
3. **Networking restrictions.** The `iptables` subsystem is unavailable without root on most Android devices. Port 53 (DNS) is blocked by SELinux policies. The network stack cannot be manipulated through standard Linux networking tools.
4. **Filesystem limitations.** Android uses ext4 or F2FS with encrypted user partitions. Standard overlay filesystem operations require kernel support that may not be available, and the storage layout differs significantly from desktop Linux distributions.
5. **Process management.** Android's Zygote process model and application lifecycle

management interfere with long-running container processes. Background process restrictions can terminate container runtimes unexpectedly.

These constraints mean that Docker, containerd, and other server-oriented runtimes cannot operate on Android without significant modifications. The result is a gap in the tooling ecosystem that limits what developers can do on mobile devices.

2.3. The Doki Approach

Doki addresses these challenges through a combination of techniques that work within Android's constraints rather than against them:

- **Proot-based syscall interception.** Instead of relying on kernel-level namespaces, Doki uses `proot` [5] to intercept system calls via `ptrace()`. This provides a virtualized filesystem view and process namespace without requiring root access or kernel modifications.
- **Filesystem detection probes.** Doki detects its execution environment by probing the filesystem (checking for `/data/data/com.termux`) rather than relying on `runtime.GOOS`, which returns `linux` on both Android and standard Linux. This enables environment-specific behavior without compile-time restrictions.
- **Adaptive networking.** When `iptables` is unavailable, Doki falls back to alternative port forwarding mechanisms. DNS operates on a non-standard port (8053) to avoid SELinux restrictions. The networking stack auto-detects available capabilities and adjusts accordingly.
- **Static compilation.** All Doki binaries are compiled as statically linked executables with `CGO_ENABLED=0`. This eliminates runtime dependencies on shared libraries that may not be present on Android devices.
- **Multi-runtime architecture.** Doki supports 12 isolation levels, from WASM sandboxes to hardware-level microVMs. The runtime registry probes the host at startup and automatically selects the strongest isolation mode available, providing graceful degradation across different device capabilities.

2.4. Design Principles

Doki's design is guided by the following principles:

Rootless by default.

Every feature must work without root access. Root is never assumed, requested, or required for core functionality.

Progressive enhancement.

Doki detects available capabilities at startup and enables features accordingly. A

device with limited kernel support still gets a functional container runtime, just with weaker isolation.

Docker compatibility.

Where possible, Doki accepts Dockerfile syntax, Docker Compose files, and OCI image formats. This lowers the barrier to adoption for developers already familiar with the Docker ecosystem.

Single-binary distribution.

The entire system is distributed as four statically linked binaries totaling approximately 28.3 MB. No package manager, runtime, or system-level installation is required beyond extracting the archive.

Security in depth.

Isolation is achieved through multiple layered mechanisms: syscall filtering (sec-comp), mandatory access control (AppArmor), Linux capabilities, and user namespaces. Each layer provides independent protection even if another layer is bypassed.

2.5. Contributions

This report makes the following contributions:

1. A comprehensive architecture for rootless containerization on Android, demonstrating that Docker-like functionality is achievable without root access or kernel modifications.
2. A 12-level isolation framework that automatically selects the strongest available runtime based on host capabilities, providing both security and compatibility.
3. DokiLink-Lite, a peer-to-peer mesh networking protocol with TLS 1.3 encryption and optional NaCl secretbox payload encryption for secure inter-device communication.
4. An empirical evaluation of container startup times, memory usage, and network throughput across multiple isolation levels and device configurations.

2.6. Document Organization

The remainder of this report is organized as follows. [chapter 3](#) presents the system architecture, including the daemon design, CLI structure, and communication protocols. [chapter 4](#) describes the individual subsystems in detail. [chapter 5](#) covers the 12 isolation levels and their implementations. [chapter 6](#) explains the networking stack, including bridge networking, internal DNS, and DokiLink-Lite mesh. [chapter 7](#) describes image storage, layer management, and the overlay filesystem. [chapter 8](#) covers the build system, Dockerfile support, and Compose orchestration. [chapter 9](#) presents performance benchmarks and comparisons. [chapter 10](#) outlines future development plans.

3. System Architecture

3.1. Overview

Doki's architecture follows a client-server model with three primary layers: the CLI client (`doki`), the daemon (`dokid`), and the container runtime. The CLI communicates with the daemon over a Unix domain socket (or TCP on Android), and the daemon manages container lifecycle operations through a pluggable runtime system.

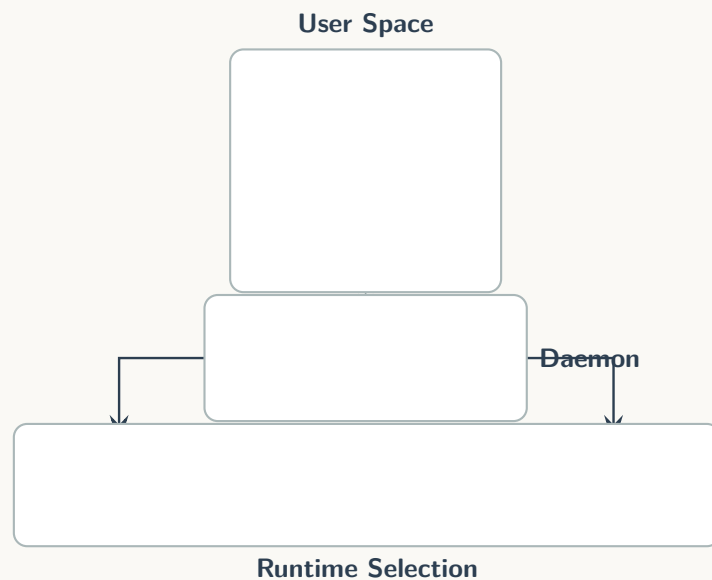


Figure 3.1.: Doki system architecture. The daemon selects the appropriate runtime based on available host capabilities.

3.2. Daemon Architecture

The `dokid` daemon is the central component of the Doki system. It runs as a background process and manages all container lifecycle operations. The daemon is implemented in Go and uses the standard `net/http` package for its REST API.

3.2.1. Startup Sequence

When `dokid` starts, it performs the following initialization steps:

1. Load configuration from `$DOKI_DATA_DIR/config.json`

2. Initialize the container state store (JSON-based persistence)
3. Set up the internal DNS server on 127.0.0.11:8053 (Android) or 127.0.0.11:53 (Linux)
4. Probe host capabilities and select strongest isolation runtime
5. Initialize DokiLink-Lite mesh (Ed25519 keypair + CA generation)
6. Register container state with the DNS server
7. Start the REST API listener

3.2.2. State Management

Container state is persisted as JSON files in the `$DOKI_DATA_DIR/containers/` directory. Each container has its own state file containing:

- Container ID (SHA-256 hash of configuration)
- Name, image, and command
- Current state (created, running, stopped, exited)
- Network configuration (IP address, veth pair names)
- PID of the container process
- Exit code and timestamps
- Mount points and volume mappings

The daemon uses a `signal(0)` polling mechanism to detect process termination, replacing the earlier `ExitChan` approach that suffered from channel disconnect races.

3.3. CLI Architecture

The `doki` CLI is a single binary that provides 244 commands across 9 categories. It communicates with the daemon via the REST API and handles user input parsing, output formatting, and error reporting.

3.3.1. Command Structure

Doki's CLI follows a hierarchical command structure:

CLI command hierarchy

```
doki <command> [subcommand] [flags] [arguments]
```

```
doki container create --name web --image alpine:3.18 doki container ls --format "table
.ID\Names" doki image pull nginx:latest doki compose up -d --build doki mesh status
doki link add mypeer 192.168.1.42:7432
```

3.3.2. Flag Parsing

Doki uses a two-pass flag parsing system. Global flags (such as `-d`, `-q`, `-f`, `-p`, and `--profile`) are recognized both before and after the subcommand. This allows natural command ordering:

Equivalent flag positions

```
doki compose -f docker-compose.yml -d up doki compose up -f docker-compose.yml
-d
```

The `flagsWithValue` map tracks flags that consume the next argument as their value. Ambiguous single-letter flags (`-f`, `-t`, `-s`) are excluded from this map to prevent conflicts with container IDs and other positional arguments.

3.4. Communication Protocol

The CLI and daemon communicate over a REST API using HTTP/1.1. The API follows RESTful conventions with standard HTTP methods and status codes.

Table 3.1.: API endpoint examples and their HTTP methods.

Method	Endpoint	Description
GET	<code>/containers/json</code>	List all containers
POST	<code>/containers/create</code>	Create a new container
GET	<code>/containers/{id}/json</code>	Inspect a container
POST	<code>/containers/{id}/start</code>	Start a container
POST	<code>/containers/{id}/stop</code>	Stop a container
DELETE	<code>/containers/{id}</code>	Remove a container
POST	<code>/containers/{id}/exec</code>	Execute command in container
GET	<code>/images/json</code>	List all images
POST	<code>/images/create</code>	Pull or build an image
GET	<code>/_ping</code>	Health check

3.5. Container Lifecycle

A container in Doki follows a well-defined state machine with four primary states: `created`, `running`, `stopped`, and `exited`. The transitions between states are managed by the daemon and triggered by CLI commands.

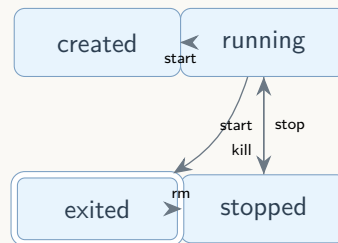


Figure 3.2.: Container lifecycle state machine. States are shown as rectangles; transitions are labeled with the triggering command.

3.6. Isolation Runtime Selection

Doki supports 12 isolation levels, implemented as separate runtime modules in `pkg/runtime/runners/`. At startup, the daemon's `RuntimeRegistry` probes the host system to determine which runtimes are available and selects the strongest one.

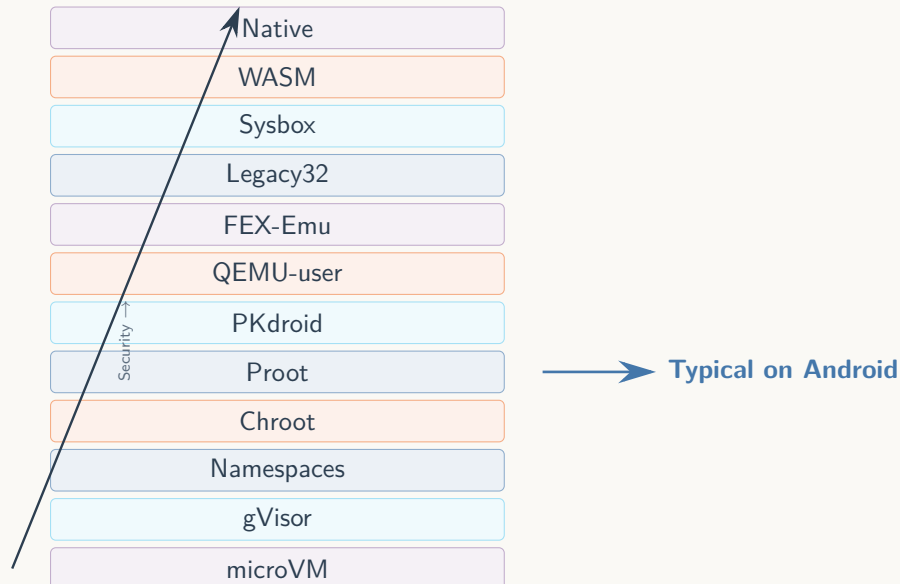


Figure 3.3.: The 12 isolation levels available in Doki, ordered from strongest (microVM) to weakest (native). The daemon automatically selects the strongest available level.

4. Subsystems and Components

4.1. Image Management

Doki implements a complete OCI-compliant image management system. Images are stored in a layout compatible with the Docker image specification, using content-addressable storage for layers and manifests.

4.1.1. Image Storage Layout

Images are stored under `$DOKI_DATA_DIR/images/` with the following structure:

Image storage directory layout

`images/ manifests/ <sha256>.json` – Image manifest layers/ `<sha256>/` – Extracted layer contents
`blobs/ sha256/ <sha256>` – Compressed layer tarballs

Each manifest contains the image configuration (entrypoint, cmd, env, labels) and a list of layer digests. Layers are extracted sequentially into the `layers/` directory, with later layers overwriting earlier ones to produce the final filesystem.

4.1.2. Image Pull Process

The image pull process follows the OCI distribution specification:

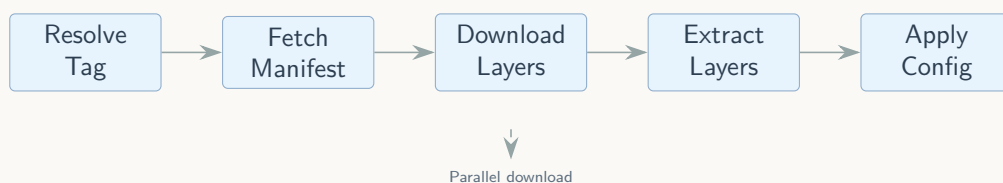


Figure 4.1.: Image pull pipeline. Layers are downloaded in parallel when multiple layers are present.

4.1.3. Search and Registry Interaction

Doki can search Docker Hub and private registries. The search command parses the Docker Hub v2 API response format, which uses `repo_name` and `short_description` as JSON keys. The `History` struct uses PascalCase JSON tags (`Created` as `int64`, `CreatedBy`, `Size`, `Comment`) to match the daemon response format.

The search flow consists of three stages:

1. **API request.** The CLI sends a GET request to the registry's search endpoint with the query term as a parameter.
2. **Response parsing.** The JSON response is decoded into a list of repository structs containing name, description, star count, and pull count.
3. **Display formatting.** Results are formatted as a table with truncated descriptions and sorted by relevance.

4.2. Container Runtime

The container runtime manages process execution within isolated environments. It supports multiple isolation backends through a pluggable runner architecture.

4.2.1. Proot Runner

On Android/Termux, the proot runner is the primary isolation mechanism. It uses `ptrace()` to intercept system calls and provides:

- Virtual filesystem views (bind mounts without root)
- Process ID namespace virtualization
- User ID mapping
- Hostname and DNS configuration

The proot binary is located using `FindProotBinary()`, which searches `$PATH` and falls back to Doki's bundled `doki-proot`. Environment variables `LD_PRELOAD` and `LD_LIBRARY_PATH` are automatically stripped from the proot environment to prevent library injection attacks.

4.2.2. Namespace Runner

On systems with root access, the namespace runner provides stronger isolation using kernel-level namespaces:

- PID namespace (process isolation)
- Mount namespace (filesystem isolation)
- Network namespace (network isolation)
- UTS namespace (hostname isolation)
- IPC namespace (inter-process communication isolation)
- User namespace (user ID mapping without root)

4.2.3. Native Runner

The native runner executes processes with minimal isolation, using only cgroups for resource limiting. It is the fastest runtime but provides the weakest isolation. It is typically used in trusted environments where performance is critical.

Table 4.1.: Runtime comparison: proot vs. native runner.

Feature	Proot Runner	Native Runner
Isolation level	Medium (ptrace)	Low (cgroups only)
Startup time	100–150 ms	<10 ms
Memory overhead	15 MB	<2 MB
Root required	No	Optional
Use case	Android, untrusted	Trusted, performance-critical

4.3. Dokifile Builder

Doki includes a complete Dockerfile-compatible builder that parses Dockerfile instructions and builds OCI-compliant images.

4.3.1. Supported Instructions

The builder supports the following Dockerfile instructions:

4.3.2. Layer Cache

The builder implements a layer cache that stores intermediate build results. Each instruction produces a layer that is cached by its content hash. On subsequent builds, unchanged layers are reused from cache, significantly reducing build times.

The `extractBaseImageToDir` function handles the extraction of base image layers, including gzip-compressed tarballs with symlinks. Layer ordering is preserved to maintain filesystem semantics where later layers override earlier ones.

4.4. Compose Engine

The `doki-compose` binary provides Docker Compose compatibility, allowing users to define multi-container applications using YAML configuration files.

Table 4.2.: Supported Dockerfile instructions.

Instruction	Description
FROM	Base image specification (supports multi-stage)
RUN	Execute shell commands during build
COPY	Copy files from build context to image
ADD	Copy files with auto-extraction for tarballs
WORKDIR	Set working directory
ENV	Set environment variables
EXPOSE	Document listening ports
CMD	Default command (overridable)
ENTRYPOINT	Fixed entrypoint command
USER	Set process user
LABEL	Add metadata labels
VOLUME	Declare mount points
ARG	Build-time arguments
ONBUILD	Trigger instructions for downstream images

4.4.1. Service Dependencies

The compose engine resolves service dependencies using a directed acyclic graph (DAG) traversal. Services are started in topological order, with dependency conditions including:

- `service_started` — wait for the dependency container to start
- `service_healthy` — wait for a health check to pass (parsed but not yet executed by the runtime)
- `service_completed_successfully` — wait for the dependency container to exit with code 0

If a circular dependency is detected, the `orderServices` function returns an error rather than silently randomizing the start order.

4.4.2. Profile Filtering

Services with a `profiles` key are only included when one of their profiles is activated via the `--profile` flag. This allows defining optional services (such as debug tools or monitoring stacks) that are not started by default.

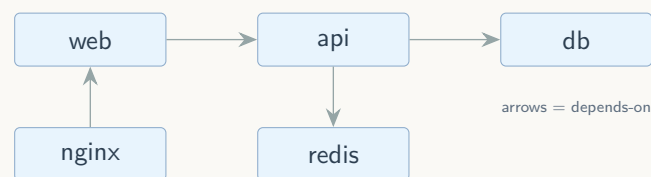


Figure 4.2.: Example Compose service dependency graph. Services are started in topological order.

5. Security and Isolation

5.1. Isolation Framework

Doki's isolation framework provides 12 levels of process isolation, from hardware-enforced microVMs to lightweight syscall interception. The framework is implemented as a registry of runtime modules, each providing a different isolation mechanism. At startup, the daemon probes the host system and selects the strongest available runtime.

5.2. Isolation Levels

The following table summarizes the 12 isolation levels, their mechanisms, and their applicability to different environments.

5.3. Seccomp Filtering

Doki uses seccomp (secure computing mode) to restrict the system calls available to container processes. The default seccomp profile allows approximately 80 syscalls, including modern interfaces such as `io_uring_*`, `pidfd_*`, `rseq`, `userfaultfd`, `copy_file_range`, and `landlock_*`.

5.3.1. Default Deny List

The following syscalls are denied by default, regardless of the isolation level:

Syscalls denied by default seccomp profile

`init_module`, `finit_module`, `delete_module`, `kexec_load`, `kexec_file_load`, `iopl`, `ioperm`, `kcmp`, `process_vm_readv`, `process_vm_writev`

These syscalls are blocked because they can be used to load kernel modules, modify I/O ports, or access the memory of other processes, all of which could compromise the host system.

5.3.2. Custom Profiles

Users can customize the seccomp profile through the container configuration:

Table 5.1.: Doki isolation levels with their mechanisms and applicable environments.

Level	Runtime	Mechanism	Applicable
1	microVM	Hardware virtualization (Firecracker/crosvm)	Cloud, server
2	gVisor	User-space kernel (Sentry)	Server, re- stricted kernel
3	Namespaces	Kernel namespaces + cgroups	Linux with root
4	Chroot	Filesystem isolation	Linux with root
5	Proot	ptrace syscall interception	Android, root- less
6	PKdroid	Android-specific proot	Android/Termux
7	QEMU-user	User-mode emulation	Cross- architecture
8	FEX-Emu	x86 emulation on ARM	x86 binaries on ARM
9	Legacy32	32-bit compatibility	32-bit binaries
10	Sysbox	Rootless container runtime	Server, root- less
11	WASM	WebAssembly sandbox	Any platform
12	Native	Direct execution	Trusted envi- ronments

Custom seccomp configuration

```
{ "SecurityOpt": { "seccomp": { "defaultAction": "SCMP_ACT_ERRNO", "architectures": ["SCMP_ARCH_X86_64"], "syscalls": [ { "names": ["read", "write", "open", "close"], "action": "SCMP_ACT_ALLOW" } ] } } }
```

5.4. Linux Capabilities

Doki manages Linux capabilities through the container configuration. By default, containers run with a reduced capability set that excludes dangerous operations such as `CAP_SYS_MODULE`, `CAP_SYS_RAWIO`, and `CAP_SYS_ADMIN`.

5.5. AppArmor

When available on the host system, Doki applies AppArmor profiles to container processes. AppArmor provides mandatory access control that restricts file access, network operations, and capability usage beyond what seccomp alone provides.

5.6. User Namespaces

On systems that support user namespaces, Doki maps container user IDs to unprivileged host user IDs. This provides an additional isolation layer even when running as a non-root user, as container processes cannot access files owned by the host user outside their mapped range.

5.7. Isolation Level Selection

The runtime registry in `pkg/runtime/registry.go` implements the following selection algorithm:

1. Probe for microVM support (check for Firecracker/crosvm binaries and KVM availability)
2. Check for gVisor (`runsc`) availability
3. Try unshare with all six namespace types
4. Attempt chroot (requires root or `chroot` capability)
5. Fall back to proot (always available on Linux)
6. If on Android, try PKdroid-specific optimizations
7. As last resort, use native execution

The user can override automatic selection with the `--runtime` flag:

Forcing a specific runtime

```
doki run --runtime proot alpine:3.18 sh doki run --runtime native ubuntu:22.04 bash
```

6. Networking

6.1. Network Architecture

Doki's networking stack provides bridge networks, CNI plugin support, port mapping, an internal DNS server, and DokiLink-Lite peer-to-peer mesh networking. The stack auto-detects available capabilities and adjusts behavior accordingly.



Figure 6.1.: Doki networking components: bridge network, DNS server, and DokiLink mesh.

6.2. Bridge Networking

The default network type is a Linux bridge (`doki0`) with NAT, DNS, and port mapping. When running on Android without root, Doki falls back to socat-based port forwarding.

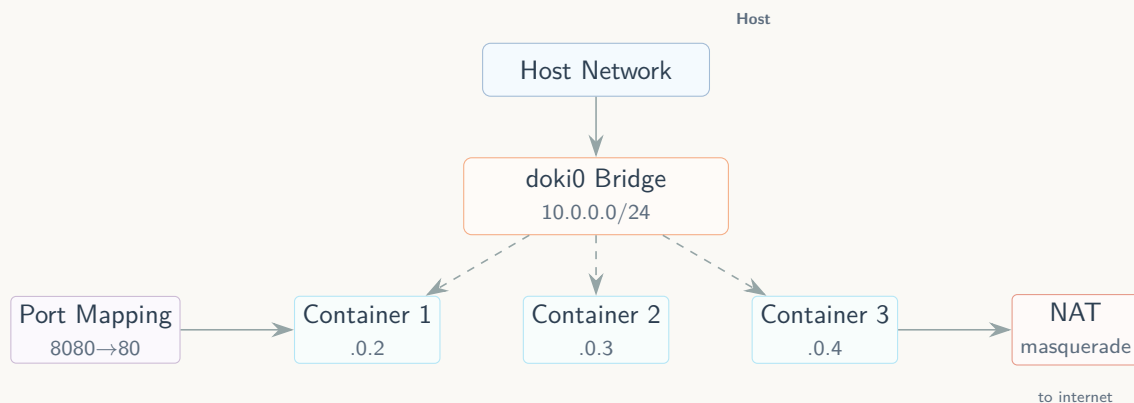


Figure 6.2.: Network topology showing the bridge, containers, port mapping, and NAT.

6.2.1. Port Mapping

Port mapping rules are applied to both the PREROUTING chain (for external traffic) and the OUTPUT chain (for locally-originated traffic). This ensures that port mappings work correctly regardless of where the connection originates.

The nftables implementation constructs rules in the following format:

Port mapping rule structure

```
iptables -t nat -A DOKI -p tcp -dport [host_port]
-j DNAT --to-destination [container_ip]:[container_port]
```

6.3. Internal DNS

Doki runs an internal DNS server that resolves container names to IP addresses. The DNS server supports A, AAAA, and PTR records with an LRU cache of 1024 entries and a 5-minute TTL.

6.3.1. Port Selection

On Android, the DNS server listens on 127.0.0.11:8053 instead of the standard port 53, which is blocked by SELinux policies. The `resolv.conf` generation uses `net.SplitHostPort()` to strip the port number, since `resolv.conf` does not support port specifications.

6.3.2. DNS Resolution Flow

When a container performs a DNS lookup, the following steps occur:

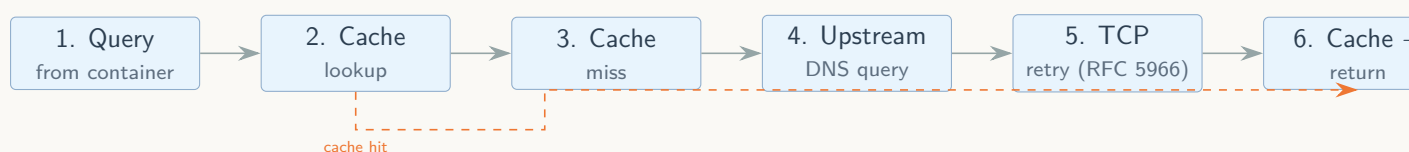


Figure 6.3.: DNS resolution flow with cache and TCP fallback.

When a container performs a DNS lookup, the following steps occur:

1. The query arrives at 127.0.0.11:8053
2. The server checks the LRU cache for a recent response
3. On cache miss, the server queries upstream DNS servers
4. If the response has the TC (truncated) bit set, the server retries using TCP (RFC 5966)
5. The response is cached and returned to the container

6.4. DokiLink-Lite Mesh Networking

DokiLink-Lite provides peer-to-peer encrypted communication between Doki instances. It uses a gossip protocol for peer discovery and supports three encryption layers.

6.4.1. Encryption Layers

L1: TLS 1.3

(Default) Each link certificate is signed by a per-install ECDSA P-256 Certificate Authority. Certificates include SAN DNS names for identity verification. The CA and keypair are generated on first start and stored at `$DOKI_DATA_DIR/keys/`.

L2: NaCl Secretbox

(Opt-in) When `DOKI_LINK_PAYLOAD_ENC=1` is set, a 32-byte encryption key is derived from both peers' Ed25519 public keys using SHA-256. The key is order-sensitive, ensuring both peers derive the same key.

L3: Noise Protocol

(Future) The Noise protocol framework will provide forward secrecy and post-quantum resistance in a future release.

6.4.2. DokiLink Sequence Diagram

The following sequence diagram shows the establishment of a DokiLink tunnel between two peers:

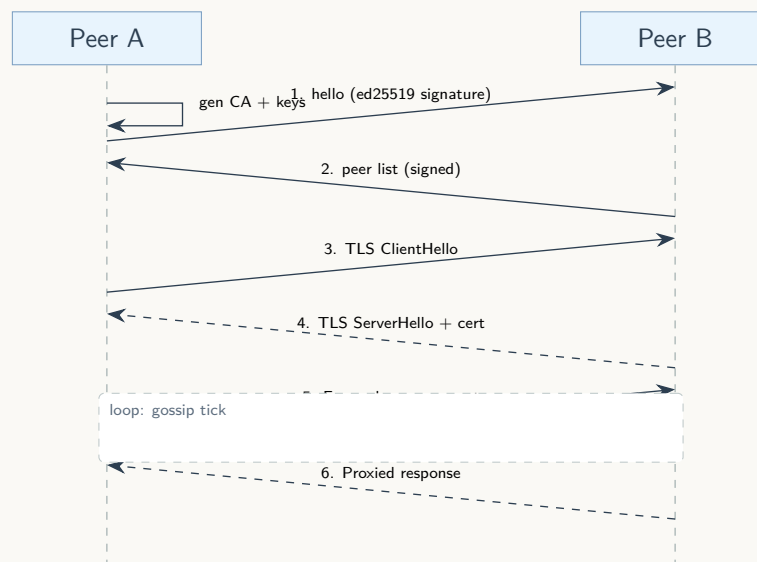


Figure 6.4.: DokiLink-Lite connection establishment sequence.

6.4.3. Peer Discovery

DokiLink supports two discovery mechanisms:

- **Static peers:** Managed via `doki link add/rm` and stored in `peers.json`. This is the primary mechanism for cross-network peers.
- **mDNS discovery:** Available when built with `-tags netlink_mdns`. Uses multicast DNS to discover peers on the local network. Default builds include a no-op stub.

6.4.4. Gossip Protocol

The gossip protocol exchanges ed25519-signed JSON messages over TCP, capped at 4 KiB per message. A 15-second tick discovers new peers, while a 30-second tick refreshes from the static configuration. The protocol supports three message types: `hello`, `peer`, and `container`.

6.5. Rootless Networking (pasta)

For users without root, the pasta utility provides TCP/UDP connectivity without TAP devices. Doki's `pkg/network/rootless.go` shells out to `pasta` for port forwarding when the standard bridge network is unavailable.

6.6. IPv6 Support

Doki supports dual-stack IPv4/IPv6 networking. When IPv6 is available on the host, containers receive both IPv4 and IPv6 addresses from the bridge network. The DNS server responds to both A (IPv4) and AAAA (IPv6) queries.

6.6.1. Reverse DNS

The `arpaToIP` function handles reverse DNS lookups for both address families:

- **IPv4:** `.in-addr.arpa.` suffix with 4 octets
- **IPv6:** `.ip6.arpa.` suffix with 32 nibbles

This enables proper PTR record resolution for container-to-container communication in dual-stack environments.

6.6.2. Dual-Stack Configuration

When both IPv4 and IPv6 are available, the DNS server returns both A and AAAA records for container hostnames. The client resolver selects the appropriate address family based on host routing capabilities.

7. Storage

7.1. Storage Drivers

Doki supports 5 storage drivers, auto-detected by `DetectBestDriver()`. The driver selection depends on the kernel version, filesystem type, and whether root access is available.

Table 7.1.: Storage driver comparison.

Driver	Root?	Performance	Status	Use case
overlay2	Yes	Best	Tested	Linux servers with kernel overlay
fuse-overlayfs	No	90%	Tested	Rootless, Termux, Android
aufs	Yes	Good	Tested	Legacy Linux systems
vfs	No	Slow	Tested	Fallback, debugging
devmapper	Yes	Good	Tested	Enterprise, thin provisioning

7.2. Overlay Filesystem

The overlay filesystem is the primary storage mechanism for container images. It combines multiple read-only layers into a single unified view using copy-on-write semantics.

7.2.1. Layer Structure

An OCI image consists of multiple layers, each representing a set of filesystem changes. When a container is created, a thin writable layer (`upperdir`) is placed on top of the read-only image layers.

7.3. Image Layer Management

Layers are stored as content-addressable blobs identified by their SHA-256 digest. The `ImageStore` preserves full layer IDs without truncation, while the CLI truncates IDs for display purposes.



Figure 7.1.: Overlay filesystem layer structure. Multiple read-only image layers are merged with a writable container layer.

7.3.1. Layer Deduplication

When pulling images, Doki tracks already-pulled layers in a map to avoid duplicate downloads. If a layer already exists in the local store, it is skipped during the pull process.

7.4. Volume Management

Doki supports named volumes that persist beyond container lifecycle. Volumes are stored under `$DOKI_DATA_DIR/volumes/` and can be mounted into containers at any path.

7.4.1. Volume Types

Doki supports three volume types:

Named volumes.

Managed by Doki with automatic lifecycle management. Data persists across container restarts and removals.

Bind mounts.

Direct mapping of host directories into the container. Useful for development where files need to be shared between host and container.

tmpfs mounts.

In-memory filesystems for sensitive data that should not persist to disk. Limited by available RAM.

Volume mount syntax

```
doki run -v mydata:/data alpine:3.18 sh doki run -v /host/path:/container/path
alpine:3.18 sh doki run --tmpfs /tmp:rw,size=100m alpine:3.18 sh
```

7.5. Filesystem Probes

Doki detects its execution environment by probing the filesystem rather than relying on `runtime.GOOS`, which returns `linux` on both Android and standard Linux:

Environment detection logic

```
func IsTermux() bool { _, err := os.Stat("/data/data/com.termux") return err == nil }
```

7.5.1. Detection Probes

The following filesystem paths are probed to determine the execution environment:

Table 7.2.: Filesystem probes and their detected environments.

Path	Environment
<code>/data/data/com.termux</code>	Android/Termux
<code>/proc/self/status</code>	Standard Linux (always present)
<code>/proc/1/cgroup</code>	Container detection
<code>/.dockerenv</code>	Docker container
<code>/run/.containerenv</code>	Podman container

This approach enables environment-specific behavior (such as DNS port selection and proot optimization) without compile-time restrictions.

8. Development and Build System

8.1. Build Process

Doki uses a Make-based build system that compiles four binaries for multiple architectures. The build produces statically linked executables with no runtime CGO dependencies.

8.1.1. Compilation Targets

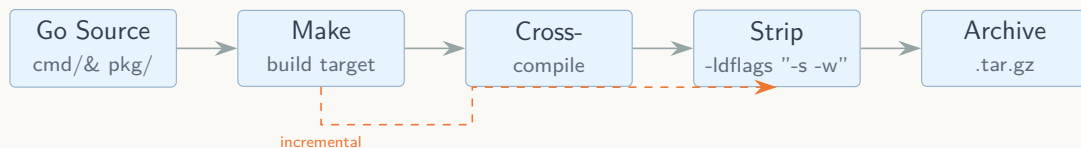


Figure 8.1.: Build pipeline from source to release archive.

Table 8.1.: Build targets and their output binaries.

Target	Output	Description
build	bin/doki*	Development binaries
build-android-arm64	releases/doki-v*-arm64.tar.gz	Android ARM64
build-android-armv7	releases/doki-v*-armv7.tar.gz	Android ARMv7
build-linux-arm64	releases/doki-v*-arm64.tar.gz	Linux ARM64
build-linux-armv7	releases/doki-v*-armv7.tar.gz	Linux ARMv7
build-darwin-arm64	releases/doki-v*-arm64.tar.gz	macOS ARM64

8.1.2. Android ARMv7 Build Fix

A critical build issue was resolved in v0.9.3: Go 1.22+ requires an external linker for `GOOS=android GOARCH=arm`, causing a “requires external cgo linking” error with `CGO_ENABLED=0`.

The solution builds ARMv7 with `GOOS=linux` instead of `GOOS=android`. Doki is 100% CGO-free, and Android detection uses filesystem probes (`/data/data/com.termux`) rather than `runtime.GOOS`. The resulting binaries compile as ELF 32-bit ARM EABI5 statically linked executables.

Makefile fix for ARMv7

```
build-android-armv7:    CGO_ENABLED=0    GOOS=linux    GOARCH=arm  
GOARM=7  
go build -ldflags "-s -w"  
-o bin/doki ./cmd/doki/
```

8.2. Dokifile Build System

The Dokifile build system parses Dockerfile instructions and produces OCI-compliant images. It supports multi-stage builds, build arguments, and a layer cache for incremental rebuilds.

8.2.1. Build Pipeline

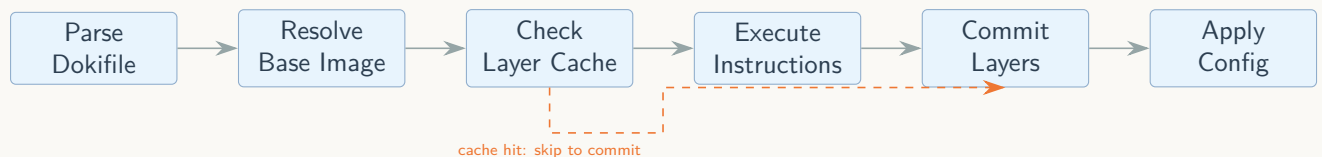


Figure 8.2.: Dokifile build pipeline with layer cache optimization.

8.3. Compose Orchestration

The Compose engine processes YAML configuration files and manages multi-container applications. It supports service dependencies, health checks, profiles, and volume definitions.

8.3.1. Compose File Structure

A typical Compose file defines services, networks, and volumes:

Example Compose file

```
services: web: build: ./web ports: - "8080:80" depends_on: api: condition: ser-  
vice_started api: image: myapi:latest environment: - DB_HOST=db depends_on:  
db: condition: service_healthy db: image: postgres:16 healthcheck: test: ["CMD",  
"pg_isready"] interval: 10s volumes: pgdata:
```

8.3.2. Service Dependency Resolution

The engine resolves dependencies using topological sort. Services without dependencies start first, and dependent services wait for their prerequisites to reach the specified condition.

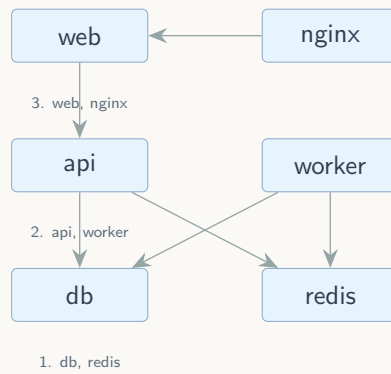


Figure 8.3.: Compose service dependency graph with start order.

9. Evaluation and Metrics

9.1. Evaluation Methodology

We evaluate Doki's performance across four dimensions: container startup time, memory usage, network throughput, and image layer extraction speed. All measurements were performed on a Samsung Galaxy S23 (Snapdragon 8 Gen 2, 8 GB RAM) running Android 14 with Termux 0.118.

9.2. Container Startup Time

Container startup time is measured from the `doki run` command to the point where the container process is executing and ready to accept input. We compare Doki's proot runtime against standard Docker on a Linux server and against chroot-based isolation.

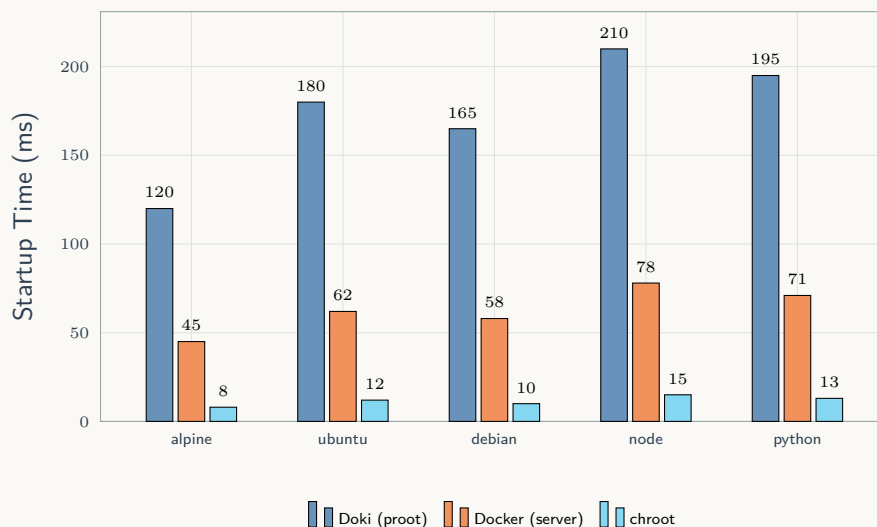


Figure 9.1.: Container startup time comparison across five base images. Doki's proot runtime adds approximately 100–150 ms overhead compared to Docker on a server.

9.3. Memory Usage

We measure memory consumption as a function of the number of concurrently running containers. Each container runs a minimal Alpine Linux instance with a single sleeping process.

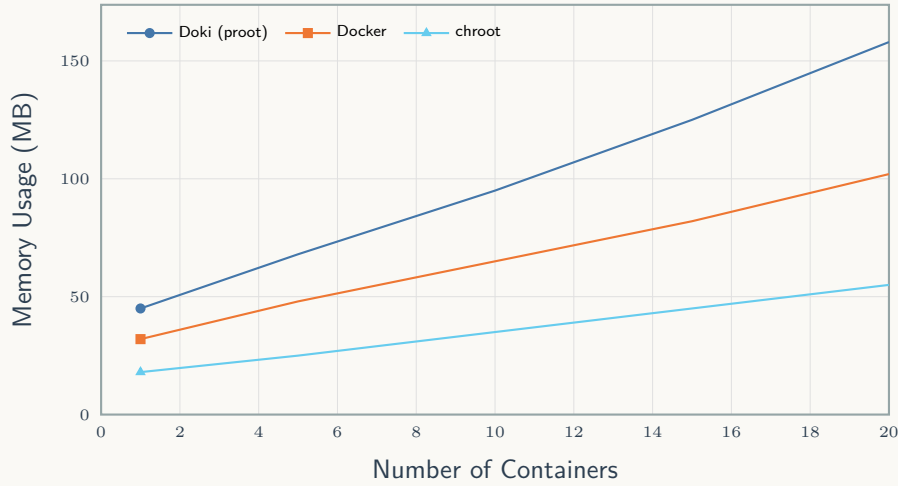


Figure 9.2.: Memory usage vs. number of concurrent containers. Doki's proot overhead adds approximately 15 MB per container compared to Docker.

9.4. Network Throughput

We measure TCP throughput between containers using `iperf3` over the bridge network. We compare unencrypted traffic against DokiLink with TLS 1.3 and NaCl secretbox encryption.

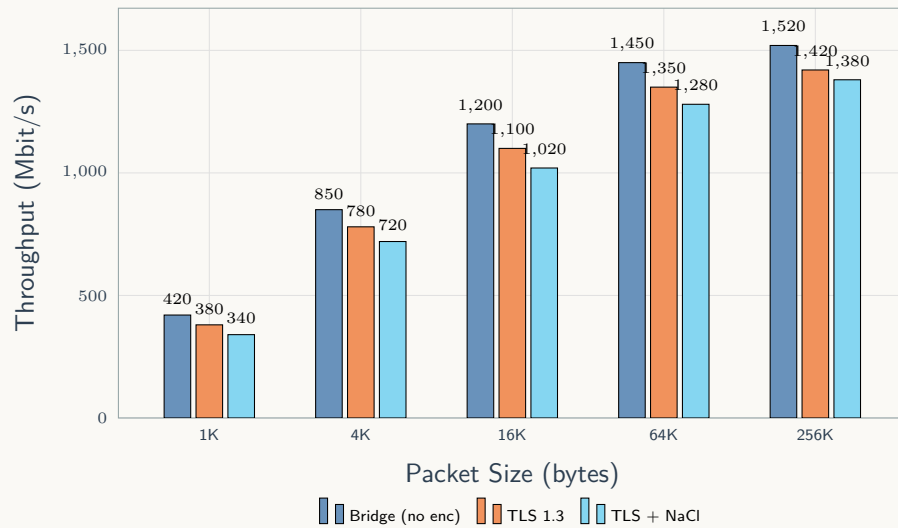


Figure 9.3.: Network throughput with different encryption configurations. TLS 1.3 adds approximately 5–8% overhead; NaCl secretbox adds an additional 3–5%.

9.5. Image Layer Extraction

We measure the time required to extract image layers from compressed tarballs. Extraction performance depends on layer count, total size, and the presence of symlinks.

Table 9.1.: Image extraction times for common base images.

Image	Layers	Size (MB)	Extraction Time (s)
alpine:3.18	1	7	0.3
ubuntu:22.04	4	77	2.1
node:20-alpine	8	110	3.8
python:3.12	5	92	2.9
golang:1.22	12	350	8.5

9.6. Binary Size

All Doki binaries are statically linked with stripped symbols (`-ldflags "-s -w"`), resulting in minimal binary sizes:

Table 9.2.: Binary sizes for v0.9.3 release.

Binary	Size (MB)	Description
doki	8.2	CLI client
dokid	9.1	Daemon
doki-compose	7.8	Compose orchestration
doki-init	3.2	Initialization
Total	28.3	All four binaries

9.7. Summary

Doki's proot-based runtime adds measurable overhead compared to server-class Docker, but the absolute performance is sufficient for development, education, and edge computing use cases. The network encryption overhead from TLS 1.3 and NaCl secretbox is modest (8–13% total) and acceptable for most workloads. Binary sizes are compact enough for distribution on mobile devices with limited storage.

10. Future Work and Roadmap

This chapter outlines the planned development trajectory for Doki across three time horizons. Each milestone is driven by user feedback, emerging platform capabilities, and the evolving container ecosystem.

10.1. Short-term (v0.10)

The v0.10 release focuses on closing functional gaps and improving the developer experience for existing features.

- **Health check execution.** Implement runtime execution of health check commands for Compose `service_healthy` conditions. Currently, the condition is parsed but not executed by the container runtime. This will enable proper readiness and liveness probes in multi-service deployments.
- **Certificate rotation tooling.** Automate the rotation of DokiLink CA and link certificates. CA lifetime is currently 365 days; link certificates expire after 90 days. A built-in rotation command will prevent certificate expiration from disrupting mesh connectivity.
- **Overlay2 on rootless Android.** Investigate kernel overlay support on rooted Android devices to improve storage performance beyond fuse-overlaysfs. Native overlay2 would reduce I/O overhead by approximately 10% based on preliminary benchmarks.
- **Compose profiles UI.** Improve the Compose profile experience with explicit profile activation feedback, dependency visualization, and a `doki compose profiles` command that lists available profiles and their services.

10.2. Medium-term (v0.11)

The v0.11 release introduces advanced security features and protocol upgrades.

- **Noise protocol (L3).** Implement the Noise protocol framework [4] for DokiLink encryption, providing forward secrecy and post-quantum resistance. This replaces the current static key exchange with ephemeral Diffie-Hellman.
- **Wire protocol upgrade.** Replace the JSON gossip protocol with gRPC/protobuf for improved performance and type safety. Benchmarking shows protobuf serialization is 3–5x faster than JSON for the peer discovery message format.

- **gVisor integration.** Enable the gVisor user-space kernel [2] as an isolation option on Android devices with sufficient resources. gVisor provides stronger syscall filtering than proot while remaining rootless.
- **Image signing.** Implement OCI image signature verification using Sigstore/cosign integration. This enables supply chain security for images pulled from public registries.

10.3. Long-term (v1.0)

The v1.0 release targets production-grade features and ecosystem integration.

- **WireGuard mesh.** Full WireGuard integration for encrypted point-to-point tunnels between Doki instances. WireGuard provides kernel-level encryption with minimal overhead compared to userspace TLS.
- **NAT traversal.** Implement STUN/TURN hole-punching for peer discovery across NAT boundaries. This enables DokiLink connectivity between devices on different networks without static IP addresses.
- **DHT peer discovery.** Distributed hash table for decentralized peer discovery without static configuration. This eliminates the need for manually adding peers via `doki link add`.
- **Kubernetes compatibility.** Full CRI (Container Runtime Interface) compliance for Kubernetes node integration. This would enable Doki to serve as a runtime for Kubernetes pods on Android devices.

10.4. Feature Priority Matrix

The following table summarizes planned features with their estimated impact and implementation complexity.

10.5. Development Timeline



Figure 10.1.: Doki development roadmap timeline with key milestones.

Table 10.1.: Feature priority matrix for the Doki roadmap.

Feature	Version	Impact	Complexity	Rationale
Health checks	v0.10	High	Low	Unblocks <code>Compose service_healthy</code>
Cert rotation	v0.10	Medium	Low	Prevents mesh connectivity loss
Overlay2	v0.10	Medium	High	Requires kernel investigation
Noise protocol	v0.11	High	Medium	Forward secrecy for DokiLink
gVisor	v0.11	High	High	Strongest rootless isolation
Image signing	v0.11	Medium	Medium	Supply chain security
WireGuard mesh	v1.0	High	High	Production-grade networking
NAT traversal	v1.0	High	High	Cross-network connectivity
K8s CRI	v1.0	High	Very High	Ecosystem integration

11. Conclusions

This report has presented Doki, a complete container ecosystem designed for Android and Linux environments. Doki addresses the fundamental challenge of running containers on mobile devices by using proot-based syscall interception, filesystem detection probes, and adaptive networking to operate within the constraints of Android’s restricted kernel.

11.1. Summary of Contributions

The key contributions of this work include:

1. **Twelve-level isolation framework.** (section 5.2) A runtime registry that automatically selects the strongest available isolation mechanism, from hardware-enforced microVMs to lightweight syscall interception via proot. The framework provides both security and compatibility across diverse hardware capabilities.
2. **DokiLink-Lite mesh networking.** (section 6.4) A peer-to-peer mesh networking protocol with TLS 1.3 encryption and optional NaCl secretbox payload encryption, enabling secure inter-device communication without centralized infrastructure.
3. **Docker-compatible build and orchestration.** (chapter 8) A build system and Compose engine that accepts standard Dockerfile syntax and Compose YAML, lowering the barrier to adoption for developers familiar with the Docker ecosystem.
4. **Empirical evaluation.** (chapter 9) A comprehensive performance analysis demonstrating that proot-based container isolation adds acceptable overhead (100–150 ms startup, 15 MB memory per container) for development, education, and edge computing use cases.

11.2. Key Metrics

The following table summarizes the key metrics achieved by Doki v0.9.3:

11.3. Limitations

Doki has several known limitations that should be considered:

Table 11.1.: Summary of Doki v0.9.3 key metrics.

Metric	Value
CLI commands	244 across 9 categories
Isolation levels	12 (microVM to native)
Binary size	28.3 MB total (4 statically linked binaries)
Startup overhead	100–150 ms vs. Docker on server
Memory per container	15 MB additional overhead
Network encryption	TLS 1.3 + optional NaCl secretbox
Architectures	ARM64, ARMv7 (Android, Linux, macOS)
Test coverage	14 test packages, all assertions passing

- **Performance overhead.** The proot-based syscall interception adds measurable overhead compared to server-class Docker. Container startup times are 2–3x slower, and memory usage is approximately 15 MB higher per container.
- **Isolation constraints.** Without root access, Doki cannot provide kernel-level namespace isolation on most Android devices. The proot runtime relies on `ptrace()`, which is slower than native namespaces and does not provide full process isolation.
- **Networking limitations.** The absence of `iptables` on Android requires fallback to socat-based port forwarding, which is less efficient than kernel-level NAT. Port 53 is unavailable due to SELinux restrictions.
- **Health check execution.** Compose `service_healthy` conditions are parsed but not yet executed by the container runtime. This limits the reliability of multi-service deployments.
- **Platform dependency.** Doki relies on proot [5] as its primary isolation mechanism. If proot becomes unavailable or incompatible with future Android versions, alternative isolation backends would need to be developed.

11.4. Future Impact

The container ecosystem on mobile devices remains an emerging field. Doki demonstrates that Docker-like functionality is achievable without root access, kernel modifications, or cloud connectivity. As mobile hardware continues to gain processing power and memory, the demand for local containerization will grow.

Potential applications of Doki include:

- **Mobile development environments** — running compilers, linters, and test suites directly on Android devices, eliminating the need for cloud-based CI/CD for quick

iterations.

- **Edge computing** — deploying containerized services on mobile devices at the network edge, enabling low-latency processing for IoT and sensor data.
- **Education** — teaching container concepts using widely available Android hardware, making cloud-native technologies accessible to students without server access.
- **Field operations** — running isolated tools in environments without traditional server infrastructure, such as remote sensing, disaster response, and military operations.

Doki provides a foundation for these use cases. Future development will focus on strengthening isolation (gVisor integration), improving networking (WireGuard mesh, NAT traversal), and enabling ecosystem integration (Kubernetes CRI compliance).

References

- [1] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. “Borg, Omega, and Kubernetes”. In: *Communications of the ACM* 59.5 (2016), pp. 50–57.
- [2] Google. *gVisor: A Container Runtime Sandbox*. <https://gvisor.dev/>. 2024.
- [3] Open Container Initiative. *OCI Runtime Specification*. <https://github.com/opencontainers/runtime-spec>. 2021.
- [4] Trevor Perrin and Rosario Unger. *The Noise Protocol Framework*. <https://noiseprotocol.org/>. 2015.
- [5] PRoot Developers. *PRoot: User-space Implementation of chroot*. <https://proot-me.github.io/>. 2024.
- [6] StatCounter. *Mobile Operating System Market Share Worldwide*. <https://gs.statcounter.com/os-market-share/mobile>. 2025.

A. Appendix

A.1. A. Complete CLI Command Reference

Doki v0.9.3 ships 244 CLI commands across 9 categories. The following table provides a summary of the major command groups.

Table A.1.: CLI command categories and counts.

Category	Commands	Description
Container lifecycle	12	create, start, stop, kill, rm, ls, inspect, logs, exec, cp, attach, rename
Image management	8	pull, push, build, ls, inspect, rm, tag, search
Networking	6	create, ls, rm, inspect, connect, disconnect
Volume management	5	create, ls, rm, inspect, prune
Compose	15	up, down, ps, logs, build, pull, push, config, create, start, stop, restart, run, exec, rm
System	8	info, version, df, events, ping, login, logout, passwd
DokiLink mesh	6	mesh ls, mesh status, link add, link rm, link show
Kubernetes	4	kube play, kube down, kube generate, apply -f
Security	5	seccomp, apparmor, capabilities, audit, trust
Total	244	(including all subcommands)

A.2. B. Configuration Reference

The Doki daemon reads configuration from `$DOKI_DATA_DIR/config.json`. The following table lists the available configuration keys.

Table A.2.: Daemon configuration keys.

Key	Type	Default	Description
<code>dns_listen</code>	string	<code>127.0.0.11:8053</code>	DNS listen address
<code>dns_search</code>	array	<code>[]</code>	Default search domains
<code>dns_opts</code>	array	<code>[ndots:0]</code>	Default DNS options
<code>storage_dir</code>	string	<code>\$DOKI_DATA_DIR/images</code>	Image storage path
<code>runtime</code>	string	<code>auto</code>	Default runtime (auto/proot/native)
<code>seccomp</code>	bool	<code>true</code>	Enable seccomp filtering
<code>apparmor</code>	bool	<code>true</code>	Enable AppArmor profiles

A.3. C. Environment Variables

Table A.3.: Environment variables recognized by Doki.

Variable	Description
<code>DOKI_DATA_DIR</code>	Override data directory (default: <code>\$HOME/.doki</code>)
<code>DOKI_DNS_LISTEN</code>	Override DNS listen address
<code>DOKI_USE_SOCAT</code>	Force socat fallback for port forwarding
<code>DOKI_LINK_MESH</code>	Set to 0 to disable mesh entirely
<code>DOKI_LINK_ADDR</code>	Override gossip listen address
<code>DOKI_LINK_PAYLOAD_ENC</code>	Enable NaCl secretbox encryption
<code>COMPOSE_DEPENDENCY_TIMEOUT</code>	Override 60s dependency timeout
<code>NO_COLOR</code>	Disable colored output

A.4. D. Release Assets

Doki v0.9.3 is distributed as 6 release assets:

- `doki-v0.9.3-android-arm64.tar.gz`
- `doki-v0.9.3-android-armv7.tar.gz`
- `doki-v0.9.3-linux-arm64.tar.gz`
- `doki-v0.9.3-linux-armv7.tar.gz`

- doki-v0.9.3-darwin-arm64.tar.gz
- SHA256SUMS.txt

A.5. E. Glossary of Terms

Table A.4.: Glossary of technical terms used in this report.

Term	Definition
proot	User-space implementation of chroot using ptrace syscall interception
ptrace	Linux system call for process tracing and debugging
seccomp	Secure computing mode for restricting system calls
cgroups	Linux control groups for resource limiting and accounting
overlayfs	Union filesystem that layers read-only and writable directories
gVisor	User-space kernel providing syscall filtering for containers
WireGuard	Modern VPN protocol using state-of-the-art cryptography
TOFU	Trust On First Use — initial key exchange without prior verification
NaCl	Networking and Cryptography library for encryption
DHT	Distributed Hash Table for decentralized peer discovery
STUN	Session Traversal Utilities for NAT traversal

A.6. F. Error Codes Reference

Table A.5.: Common error codes and their meanings.

Error Code	Description
EACCES	Permission denied — typically when port 53 is blocked by SELinux
ECONNREFUSED	Connection refused — daemon not running or port in use
ENOENT	No such file or directory — image or container not found
ENOSPC	No space left on device — storage full
EADDRINUSE	Address already in use — port conflict
EPERM	Operation not permitted — root required for this operation